Qwen

From Rust Types to AI Contracts: Architecting a Safe and Verifiable Framework for N64 Game Development

The Technical Interoperability Layer: A Schema-Governed Contract System

The technical foundation of the proposed AI-orchestrated development framework is its interoperability layer, which functions as a rigid, verifiable communication protocol between disparate components: the AI-Chat interface, a suite of Rust-based command-line interfaces (CLIs), and the continuous integration (CI) pipeline managed by GitHub Actions [46]. This layer is explicitly prioritized as the central pillar of the system, designed to mitigate the inherent risks of agentic workflows by constraining all interactions to well-defined, machine-readable contracts [55] [60]. The core principle is one of strict separation and governance: the AI never interacts directly with raw files, shell commands, or sensitive project states; instead, it emits small, typed JSON structures that represent its intentions [7]. These structures are then validated, interpreted, and executed by deterministic Rust tools, creating a predictable and auditable chain of operations [22]. This design choice directly addresses common failure modes in conversational programming, where ambiguity and hallucination can lead to unpredictable outcomes [16].

The cornerstone of this system is the tight coupling between Rust types and their corresponding JSON Schema definitions. This is achieved primarily through the use of the `schemars` crate, which provides derive macros to automatically generate JSON Schema metadata from Rust structs [11] [37]. This process creates a powerful bidirectional relationship that ensures consistency across the entire toolchain. When a developer modifies a Rust struct, such as `pub struct BuildContract { ... }`, the `#[derive(JsonSchema)]` attribute ensures that a corresponding `buildcontract.schema.json` file is generated as part of the compilation process. This schema serves as the official contract for any file or message conforming to that structure. Conversely, the Rust application can use libraries like `serde_json` to deserialize incoming JSON data into these strongly-typed Rust instances. This

deserialization process inherently validates the input against the schema, rejecting malformed or structurally incorrect data before it can ever be processed, thus preventing a class of runtime errors at the earliest possible stage. This symbiotic relationship between code and schema definition is critical for maintaining integrity, especially in an environment where a significant portion of the input originates from an untrusted source—the generative AI model.

The enforcement of this schema discipline is not left to convention but is codified within the CI/CD pipeline through a dedicated tool, provisionally named `schema-guard`. This tool acts as an automated gatekeeper, executing a multi-step validation process on every pull request or commit that introduces potential AI-generated content [1] [4]. The first step of the `schema-guard` process is to regenerate all JSON schemas from the current state of the Rust codebase. This ensures that the set of valid schemas always reflects the latest definitions in the source code, preventing drift between the expected contract and the actual implementation. Following regeneration, the tool proceeds to validate a comprehensive list of target files against their respective schemas. These files are diverse and form the complete vocabulary of the system's contracts, including AI-generated specifications like `BuildContract`, `PatchSpec`, and `RomLayout`; constraint files such as `N64Constraints` and `NesConstraints`; configuration recipes like `RetroRecipe`; and session-related artifacts like `SessionProfile` itself. The final and most critical action of `schema-guard` is to enforce a "fail fast" policy [25]. If any validation check fails—whether due to a missing required field, a type mismatch, or an invalid value—the CI job immediately terminates with a non-zero exit code, preventing any downstream, resource-intensive operations from proceeding [47]. This prevents wasted computational resources on building a ROM or running an emulator with invalid inputs and provides immediate, clear feedback to the developer or the AI agent about the nature of the error [45]. This practice is a cornerstone of robust engineering and is particularly essential in an agentic workflow, where a single incorrect output from the AI could otherwise trigger cascading failures throughout the build and test pipeline [35].

To further enhance the utility of the CI pipeline, the framework introduces two specialized data structures for providing rich, actionable feedback to the AI: `CiFailure` and `CiDigest`. While `schema-guard` provides a binary pass/fail outcome, these objects transform the CI process into an active participant in a closed-loop learning system. A `CiFailure` is a standardized JSON object designed to capture the details of any failed step in the CI run. It includes essential metadata fields such as the name of the affected crate or target, a `kind` field that categorizes the type of failure (e.g., `SchemaViolation`, `CompileError`, `BudgetOverflow`, `PatchSafety`, `DeterminismViolation`, `TestFailure`), a human-readable `message` summarizing

the issue, and often links to detailed log files for deeper investigation [47]. The `CiDigest` is a higher-level summary that aggregates the results of the entire CI run, containing a list of zero or more `CiFailure` entries. This digest becomes a primary artifact produced by each CI execution and is uploaded as an output of the workflow [48]. Its existence allows the AI to programmatically inspect the outcome of its proposal. For instance, after submitting a `PatchSpec`, the AI can query the associated `CiDigest` and find a `CiFailure` with `kind: "PatchSafety"` and a message detailing a patch overlap, giving it explicit, structured information on what went wrong. This transforms the CI from a passive validator into an active tutor, enabling the AI to adapt its behavior and formulate a revised proposal in subsequent turns, a pattern analogous to human-in-the-loop plan-and-execute agents [18] [52]. This structured feedback loop is fundamental to steering the AI towards producing correct and effective code and configurations, moving beyond simple correctness to achieving functional and stylistic alignment with project invariants [51].

The table below summarizes the key components of the technical interoperability layer and their roles within the framework.

Component	Description	Key Tools / Technologies	Role in the Workflow
Rust Types	Core data structures defining the shape of all contracts, constraints, and configurations.	`struct` definitions, `schemars` crate.	The authoritative source of truth for all data models in the system.
JSON Schemas	Machine-readable definitions automatically generated from Rust types, describing the valid structure of JSON documents.	`schemars::JsonSchema` derive macro.	Provides a formal contract for serialization/deserialization and validation.
`schema-guard`	A CI tool that enforces schema discipline by validating all relevant JSON files against their generated schemas.	Custom CLI tool built with Rust.	Acts as a mandatory gate, failing the CI run if any contract is invalid, thereby enforcing the "fail fast" principle.
`CiFailure`	A standardized JSON object capturing the details of a single failed CI step, including kind, message, and location.	Custom Rust struct derived `JsonSchema`.	Provides granular, actionable feedback to the AI about the cause of a failure.
`CiDigest`	A summary of a CI run's outcome, containing a list of `CiFailure` objects and other status information.	Custom Rust struct derived `JsonSchema`.	Serves as the primary result artifact passed back to the AI for analysis and decision-making.

This schema-governed contract system forms the bedrock upon which all other functionality, from patch synthesis to budget planning, is built. It creates a safe, predictable, and highly controllable environment where the AI's creative and problem-solving capabilities can be harnessed without compromising the integrity of the project. By making the rules of engagement explicit, machine-readable, and strictly enforced, the framework lays the groundwork for a sophisticated and reliable AI-assisted development paradigm.

AI Orchestration and Conditioning: Steering Agentic Behavior with Context and Contracts

While the technical interoperability layer provides the necessary infrastructure for communication, the AI orchestration layer is responsible for the intelligent management of that communication. It defines how the AI interacts with the system, makes decisions, and adapts its behavior over time. This is achieved not through free-form prompting, but by embedding the AI within a structured, context-aware environment governed by contracts, invariants, and a continuous feedback loop. At the heart of this orchestration model is the concept of "session conditioning," a mechanism that treats the AI's proposals as candidates to be scored against a set of predefined rules, effectively turning the AI into a constraint-solver operating within the project's defined boundaries [53]. This approach is designed to guide the AI toward producing outputs that are not only syntactically correct according to the JSON schemas but also semantically and pragmatically appropriate for the ongoing development task [16].

The contextual awareness of the AI is provided by the overarching "Schema-First Designer" layer, which is realized through a comprehensive Knowledge Graph (KG). This graph serves as the system's collective memory and semantic map, allowing the AI to understand its own capabilities and the state of the project. The KG is composed of two primary entities: `SystemNode` and `FeatureEntry` [39]. A `SystemNode` represents a discrete piece of functionality within the ecosystem, whether it's a Rust crate, a CLI tool, an emulator, or a core library component. Each node is heavily annotated with metadata that describes its properties and responsibilities. This metadata includes `roles` (e.g., `EditableConfig`, `DeterministicCore`, `SchemaOwner`, `CliTool`, `Emulator`), `provides` (a string identifier of the capability it offers, such as `Nintendo64.Rom.LayoutOracle`), `schemas` (the file paths of the JSON schemas it governs), and `invariants` (references to the rules and constraints it must adhere to) [39]. Furthermore, the system implements a symbol extractor tool that parses the Rust crates in the workspace, collects public functions and types, and attaches them to their corresponding `SystemNode`s, creating a searchable, navigable documentation layer that connects abstract concepts to concrete code [9].

Complementing the `SystemNode` is the `FeatureEntry`, which serves as a bridge between high-level, human- and AI-understandable concepts ("superpowers") and the underlying system components. Each `FeatureEntry` in a central manifest file (e.g., `knowledgegraph/features.sonia.json`) maps a feature name like `starzip.romlayout` or `patch-synthesizer` to the relevant `SystemNode` IDs,

specifies the schemas it consumes or produces, points to example JSON specifications, and assigns tags (e.g., `Nintendoor64`, `PatchSynthesizer`, `BudgetPlanner`) that can be used for querying [49]. This structure allows the AI to reason about tasks at a conceptual level. For example, when asked to modify a ROM file, the AI can query the KG for all nodes tagged with `PatchSynthesizer` or `RomLayout`. The KG would return the relevant `SystemNode` for `starzip-cli rom-query` and provide the path to the `RomLayout` and `PatchSpec` schemas. This transforms the AI from a general-purpose text generator into a domain-specific expert with precise, actionable knowledge about how to accomplish the task by orchestrating the correct tools with the correct data formats [52].

The `session_conditioning.rs` module is the engine that drives the AI's decision-making process, acting as a real-time scoring engine for proposals [16]. When the AI generates a new JSON contract (e.g., a `PatchSpec` or a `StealthProfile`), this module evaluates it against the session's invariants before it is ever committed or executed. These invariants are divided into two categories: hard and soft. Hard invariants are non-negotiable boolean conditions that must be satisfied for a proposal to be considered acceptable. Examples include constraints like "ROM size must be less than 32 MiB" or "deterministic crates must not use `rand::thread_rng`" [39]. A proposal that violates any hard invariant is assigned a score of negative infinity, instantly marking it as unacceptable. Soft invariants, on the other hand, represent preferences or stylistic guidelines. These might include adherence to naming conventions, code style rules, or architectural best practices. Each soft invariant has an associated weight, and the proposal's score is adjusted based on how well it aligns with these preferences [51]. The overall scoring function can be formally expressed as:

$$\text{score}(p) = \begin{cases} -\infty & amp;\text{if any hard invariant } h_i(p) \text{ is false,} \\ \sum_j w_j \cdot s_j(p) & amp;\text{otherwise.} \end{cases}$$

Where p is the proposal, $h_i(p)$ is the evaluation of hard invariant i, $s_j(p)$ is the satisfaction score of soft invariant j (a value between 0 and 1), and w_j is its weight. To make this process efficient and interactive, the system exposes a CLI tool that allows the AI to perform a "dry-run scoring" query on a proposed change *before* it writes any files to disk [53]. This allows the AI to gauge the quality of its ideas in real-time and refine them iteratively, significantly reducing friction and improving the efficiency of the development loop.

The entire orchestration model operates as a closed-loop, iterative process, often referred to as a plan-and-execute cycle [35]. This workflow can be broken down into distinct

phases: 1. **Proposal:** The AI, guided by the current `SessionProfile` (which contains information about the repository, branch, active features, and invariants) and the context from the KG, formulates a new JSON contract. This could be a `BuildContract` to add a new feature, a `PatchSpec` to fix a bug, or a modification to a `N64Constraints` file. 2. **Validation and Execution:** The proposed contract is committed to a temporary branch, triggering a GitHub Actions workflow. The workflow begins with the `schema-guard` to ensure the syntax is correct. If valid, subsequent steps execute the contract using the appropriate Rust CLIs (e.g., `gamemodeai-build` to process a `BuildContract`). During execution, various tools may produce intermediate artifacts like `N64AssetManifest` or `BudgetReport`. 3. **Telemetry and Feedback Generation:** After execution, the CI job runs parsers to analyze the logs and outputs from each step. These parsers map compiler errors, test failures, budget violations, and patch safety errors into structured `CiFailure` objects. All of these objects are aggregated into a `CiDigest` artifact, which is uploaded to the CI run. 4. **Adaptive Response:** The AI retrieves the `CiDigest` from the completed CI run. It analyzes the `CiFailure` entries to understand what went wrong. Based on this empirical feedback, the AI adjusts its internal reasoning and formulates a new, revised proposal for the next turn of the loop. For example, if the `CiDigest` indicates a `BudgetOverflow` for texture VRAM, the AI might propose downsampling a texture or merging two smaller textures into one, generating a new `PatchSpec` to enact that change.

This iterative, feedback-driven loop is the core of the framework's intelligence. It allows the AI to learn from its mistakes and progressively refine its proposals until they successfully satisfy all hard and soft invariants. The `SessionProfile` is continuously updated with the latest CI status, ensuring that the AI's understanding of the project's state is always current. This creates a dynamic and responsive development environment where the AI is not just a code generator but an active collaborator, capable of debugging, optimizing, and adapting its behavior based on the tangible consequences of its actions.

The Safe Patch Synthesizer: Enforcing Safety Through High-Level Abstraction and Interval Arithmetic

The Safe Patch Synthesizer is a top-priority superpower designed to address one of the most perilous tasks in game modding and development: modifying a console ROM. Traditional methods often involve direct manipulation of byte offsets, which is notoriously error-prone and carries a high risk of corrupting the ROM, rendering it

unplayable. The synthesizer mitigates this risk through a multi-layered defense strategy that combines high-level abstraction, formal verification, and rigorous pre-flight checks. This system elevates the AI's role from a low-level hex editor to a high-level patch author, constrained by a robust safety framework that operates entirely within the established schema-first interoperability layer.

The foundation of the synthesizer is the `PatchSpec` and its constituent `PatchEdit` variants, which together form a high-level language for describing changes to a ROM [6]. Instead of specifying raw byte ranges, the AI proposes edits using logical operations such as `ReplaceFile`, `BootHook`, `JsonPatch`, or `RawIntervalPatch`. For example, to add a new texture, the AI would not need to calculate its exact position in the ROM; it could simply propose a `PatchSpec` that includes a new `ArtifactSpec` containing the texture data and a directive to insert it into a specific logical file path within the game's asset layout. This abstraction away from physical memory addresses is the first and most crucial layer of safety. The validity of the `PatchSpec` itself is guaranteed by the `schema-guard` CI tool, which ensures that the proposal conforms to the `patchspec.schema.json` before any synthesis or application takes place [9].

Once a syntactically valid `PatchSpec` is approved, the synthesizer compiles it into a concrete ROM modification. This process involves cross-referencing the logical edits in the `PatchSpec` with the `RomLayout`, another critical contract in the system. The `RomLayout` provides a detailed map of the ROM's structure, defining segments, their sizes, memory-mapped regions (like VRAM), compression schemes, and, most importantly, mutability flags [20]. A segment marked as mutable is designated by the developer as a safe area for the synthesizer to apply patches. The synthesizer uses the `starzip-cli rom-query` API, which is exposed as a `SystemNode` in the Knowledge Graph, to interrogate the `RomLayout` and determine the permissible areas for modification [39].

The second, and perhaps most innovative, layer of safety is the application of interval arithmetic to model the ROM and the proposed patches. The entire ROM is conceptualized as a single large interval, `[0, N)`, where N is the total size of the ROM in bytes. Each individual patch edit, derived from a `PatchEdit` in the `PatchSpec`, is then modeled as its own sub-interval, $I_i=[o_i,o_i+\ell_i)$, where o_i is the origin (start offset) and ℓ_i is the length of the patch payload. The synthesizer enforces two fundamental safety axioms through this mathematical model: 1. **Containment in Mutable Segment:** For a given patch interval I_i, there must exist a mutable segment $S_j=[s_{start},s_{end})$ in the `RomLayout` such that the entire patch interval is contained within it. Mathematically, this means $s_{start}\leq o_i$ and $o_i+\ell_i\leq s_{end}$. This ensures that no patch can accidentally

overwrite critical boot code, headers, or immutable assets. 2. **Pairwise Disjointness:** For any two distinct patches i and k, their intervals must not overlap. This is expressed as $I_i \cap I_k = \emptyset$ for all $i \neq k$. This prevents multiple AI-generated edits from conflicting with one another and corrupting the same region of the ROM.

These interval checks are implemented efficiently using sorted arrays or more advanced data structures like interval trees to detect overlaps and enforce the containment inequalities in logarithmic time relative to the number of existing patches [40]. Any `PatchSpec` that violates these axioms is rejected outright. Crucially, the rejection is not silent; it produces a structured diagnostic error, such as `ConstraintError::PatchOverlap` or `ConstraintError::RomBudget`, which is then translated into a `CiFailure` and fed back to the AI via the `CiDigest` [26]. This gives the AI precise, unambiguous information about the nature of the violation, allowing it to revise its proposal correctly.

Before any patching logic is handed off to the `starzip` binary, the synthesizer performs a series of pre-flight safety validations. These checks serve as a final reality-check to prevent issues that might arise during the synthesis process. For instance, the system verifies that the cumulative size of all patch payloads for a given segment does not exceed the capacity of that segment. It also cross-checks patch payloads against segment capacities and forbids overlapping boot regions, recompute checksums [26]. These pre-flight routines are encapsulated in a strongly-typed bridge, `sonia_bridge.rs`, which sits between the high-level `PatchSpec` and the `ArtifactSpec` that `starzip` ultimately consumes. This bridge performs helper validations, checking for the existence of referenced files and ensuring payload sizes are within bounds before the `starzip` tool is even called [23]. This layered approach—from high-level abstraction to formal verification to pre-flight checks—creates a deeply defensive system that minimizes the possibility of generating a harmful patch.

The following table outlines the key components and their interaction within the Safe Patch Synthesizer workflow.

Component	Description	Interaction and Purpose
`PatchSpec`	A high-level, schema-governed JSON structure representing a collection of desired ROM modifications.	The AI's primary output for proposing changes. Its schema is validated by `schema-guard` [9].
`PatchEdit` Variants	Specific instructions within `PatchSpec`, such as `ReplaceFile` or `JsonPatch`, that describe logical changes [6].	Abstract away from low-level byte manipulation, forcing the AI to think in terms of assets and data structures.
`RomLayout`	A schema-governed JSON structure defining the ROM's segmented memory map, including mutability flags [20].	Provides the ground truth about where patches can and cannot be applied. Queried by `starzip-cli rom-query`.
`starzip-cli rom-query`	A CLI tool that exposes a programmatic API to query the contents and structure of a `RomLayout` [39].	Allows the synthesizer to dynamically discover segment boundaries and properties at synthesis time.
Interval Arithmetic Model	A mathematical model treating the ROM and patches as numerical intervals `[start, end)`.	Enforces the core safety axioms: patch containment within mutable segments and absence of patch overlaps [40].
Pre-flight Validator (`sonia_bridge`)	A Rust module that performs final checks on a synthesized patch before it is passed to `starzip`.	Validates file existence, payload sizes vs. segment capacity, and other static constraints to prevent synthesis errors [23].
Structured Diagnostics	Error types like `ConstraintError::PatchOverlap` that provide clear, machine-readable feedback.	Translated into `CiFailure` objects, giving the AI precise information about why a proposal was rejected [26].

By combining these elements, the Safe Patch Synthesizer becomes a powerful yet controlled tool. It empowers the AI to perform complex ROM modifications safely and reliably, automating a task that traditionally requires deep, manual expertise. The strict adherence to the schema-first principle ensures that every step of the process is transparent, verifiable, and integrated into the larger CI/CD feedback loop, making the synthesizer a cornerstone of the framework's commitment to safety and predictability.

The Hardware-Aware Budget Planner: Automating N64 Resource Constraints

Developing games for the Nintendo 64 is an exercise in constant negotiation with severe hardware limitations. Constraints on ROM cartridge size, onboard RAM, texture memory (VRAM), audio sample space, and CPU cycles per frame are not merely suggestions; they are absolute limits that dictate nearly every design decision. Historically, managing these budgets has been a manual, error-prone, and highly iterative process, often leading to late-stage redesigns when a feature proves too expensive to implement. The Hardware-

Aware Budget Planner superpower is designed to automate and formalize this negotiation, transforming it from a black art into a deterministic, data-driven discipline. It achieves this by creating a closed-loop system that defines constraints, tracks asset consumption, enforces limits automatically in CI, and even suggests optimizations when budgets are exceeded.

The foundation of the planner is the formalization of hardware limits into a structured data format known as `N64Constraints` [20]. This Rust struct, whose schema is rigorously defined and version-controlled, captures the essential parameters of a target N64 hardware profile. This includes maximum ROM capacity (e.g., 16, 32, or 64 MiB), available main RAM (e.g., 4 or 8 MiB), and, critically, separate pools for different asset classes. For example, it defines the maximum size of the texture pool in VRAM, the audio pool for uncompressed samples, and script/data pools for in-game logic and data. It also includes a CPU budget, typically expressed as a maximum number of clock cycles allowed per target frame rate (e.g., 60 FPS) [12]. By encoding these constraints in a machine-readable format, the system ensures that everyone—developer, AI, and CI pipeline—shares a single, unambiguous definition of the project's resource envelope.

The second component of the system is the `N64AssetManifest`, a detailed record of every asset's resource consumption. As assets are created or modified, the relevant tooling—such as packers, compilers, or layout engines—must emit a manifest entry for them. An entry in the `N64AssetManifest` would detail an asset's segment in the final ROM, its classification (e.g., texture, audio, script), its compressed and decompressed sizes, its footprint in VRAM, and an estimated CPU cost for loading or execution [62]. This manifest provides the granular data needed to accurately compute usage against the high-level constraints defined in `N64Constraints`.

With both the constraints and the asset consumption data defined, the `starzip-budget` command-line tool acts as the automated enforcement engine. It is designed to be a standard gate in the CI pipeline, invoked as part of the N64 vertical slice workflow [57]. The command-line invocation is straightforward: `starzip-budget --constraints constraints.n64.json --manifest assets.n64-manifest.json`. The tool reads both files, computes the total usage for each resource dimension (ROM, VRAM, Audio Pool, etc.), and compares it against the maximums specified in the `N64Constraints`. If the project is within budget, the tool exits with a zero status code, allowing the CI pipeline to proceed. However, if any budget is exceeded, the tool's behavior is twofold. First, it returns a non-zero exit code, causing the entire CI job to fail immediately. Second, it generates a detailed `BudgetReport` in JSON format and saves it as an artifact of the CI run [27]. This report is far more informative than a

simple "over budget" message. It provides a global summary of the overrun and, crucially, a detailed breakdown by asset class and segment, highlighting exactly which assets are contributing most to the overflow [51]. This `BudgetReport` is then parsed by the CI pipeline and translated into a `CiFailure` with a `kind` of `BudgetOverflow`, providing the AI with the specific, actionable information it needs to debug the issue.

Beyond reactive enforcement, the research plan also explores proactive budget management through mathematical optimization. The system aims to treat the budget constraints as a set of inequalities that define a feasible polytope in a multi-dimensional resource space [12]. Research objectives include defining approximate closed-form size formulas for assets—for instance, the VRAM size of a texture can be approximated by its dimensions and bit-depth, plus palette size. With these formulas, the system can implement a small planner that evaluates candidate changes (e.g., lowering texture quality, simplifying a model, cutting a redundant sound effect) as variables. Using a greedy heuristic that calculates the "value lost per byte/cycle saved," the planner can suggest concrete, quantifiable actions to bring the project back within budget. These suggestions would be exposed to the AI and developer via a `PlannerResult` JSON schema, effectively integrating a rudimentary optimization engine into the development workflow. This shifts the paradigm from constantly reacting to budget overruns to actively planning for them, empowering developers to make informed trade-offs between creative ambition and technical feasibility from the earliest stages of development.

The table below summarizes the workflow of the Hardware-Aware Budget Planner.

Stage	Component	Description	Purpose
1. Define	`N64Constraints`	A JSON schema and Rust struct defining all hardware resource limits (ROM, RAM, VRAM, CPU).	Establishes the single source of truth for the project's budget envelope.
2. Track	`N64AssetManifest`	A JSON file generated by tooling that lists every asset and its precise resource consumption (size, VRAM, CPU).	Provides the granular data needed to compute actual usage against the defined constraints.
3. Enforce	`starzip-budget`	A deterministic CLI tool that consumes `N64Constraints` and `N64AssetManifest` to validate budgets.	Acts as a mandatory CI gate, failing the build if any budget is exceeded and providing a detailed `BudgetReport`.
4. Inform	`BudgetReport`	A structured JSON artifact produced by `starzip-budget` that details usage, overages, and contributions by asset class.	Gives developers and the AI precise, actionable data on the root causes of budget overflows.
5. Optimize (Future)	Knapsack-like Planner	A research-oriented component that uses mathematical optimization to suggest asset-level changes to resolve overages.	Proactively suggests trade-offs to help the project get back on budget, shifting from reactive correction to proactive planning.

By automating the tedious and unforgiving task of resource accounting, the Hardware-Aware Budget Planner frees up cognitive load for the development team. It allows them to focus on creative problem-solving and gameplay design, knowing that the system will catch and diagnose any violations of the harsh realities of N64 hardware. This automation is a critical step in making AI-assisted N64 development a viable and productive endeavor.

The Emulator Scenario Director: Validating Gameplay Logic in Deterministic Telemetry Loops

While the Safe Patch Synthesizer and Budget Planner operate on the structural and resource aspects of a game project, the Emulator Scenario Director superpower extends the validation loop to the game's dynamic, behavioral core. Its purpose is to enable the AI and the development framework to reason about and verify gameplay logic, mechanics, and player experience in a systematic, repeatable manner. It achieves this by orchestrating headless emulators to run predefined scenarios and capturing detailed telemetry, which is then fed back into the CI pipeline as structured `CiFailure` and `CiDigest` objects. This bridges the gap between compile-time correctness and run-time quality, allowing the AI to participate in testing and debugging the game's functional and experiential qualities.

The foundation of the Scenario Director is the `ScenarioSpec`, a user-friendly JSON schema that defines a complete test case. A `ScenarioSpec` file describes a fully scripted sequence of events, starting from an initial game state (e.g., a specific map loaded, player equipped with certain weapons, enemy positions set, stealth parameters configured) [21]. It then defines a sequence of scripted player inputs, represented as bitmask codes corresponding to button presses over a series of frames. This allows for the precise recreation of complex player actions or edge-case maneuvers. Crucially, the spec also declares a list of telemetry signals to monitor during the scenario's execution, such as guard awareness values, lock-on events, hit/miss indicators, and character positions. Finally, it includes a set of assertions that define the expected outcome of the scenario. For example, an assertion might state, "Guards must not detect the player within the first 500 frames" or "The player's projectile must strike the target boss." This declarative approach allows non-programmers to define complex gameplay tests simply by editing a JSON file.

The execution of these scenarios is handled by small, custom Rust harnesses built around headless emulators like Conk64 (for N64) or other suitable emulators for NES/SNES [5]. These harnesses are themselves registered as `SystemNodes` in the Knowledge Graph, exposing their capabilities to the AI. When triggered by a CI workflow, the harness performs the following steps: 1. **Initialization:** It loads the initial game state specified in the `ScenarioSpec`. 2. **Deterministic Execution:** It runs the emulator for the duration of the scenario, applying the scripted inputs on each frame. The use of a headless, deterministic emulator ensures that the outcome of the scenario is perfectly reproducible across different machines and different runs, which is essential for reliable testing [50]. 3. **Telemetry Capture:** Throughout the execution, the harness reads memory addresses or communicates with the emulator to capture the values of the telemetry signals declared in the `ScenarioSpec`. 4. **Assertion Evaluation:** Upon completion, the harness evaluates the captured telemetry against the assertions defined in the specification. It determines whether the scenario passed or failed and records the specific reasons for any failures (e.g., "awareness threshold of 0.8 was breached at frame 153").

The final output of the harness is a `ScenarioResult` JSON object that summarizes the outcome. This result is then integrated directly into the CI pipeline's feedback mechanism. Just like a compiler error or a budget overflow, a failed scenario test is parsed and converted into a `CiFailure` object. This object is added to the `SessionProfile.ciStatus` and included in the final `CiDigest` artifact. This seamless integration means that the AI can now reason about gameplay bugs in the exact same way it reasons about build failures. If the AI proposes a change to a stealth parameter in `StealthParams.json`, and the subsequent CI run shows a `CiFailure` from the Scenario Director indicating that guards are now detecting the player too easily, the AI receives this information as structured data. It can then adjust its proposal, perhaps by tweaking the light exponent or visibility distance parameters, and resubmit the change.

Furthermore, the data collected from successful scenario executions opens up possibilities for more advanced property-based testing and mathematical modeling of gameplay systems [17]. For instance, the simulator can be treated as a function, $f(S_0, I_{0..T})$, that maps an initial state and a sequence of inputs to a trajectory of awareness values or lock-on times. By running thousands of variations of a scenario with slightly different initial parameters, the system can gather data to create curves and surfaces that model the game's stealth detection algorithms or lock-on physics. This data-driven approach allows for the creation of more robust tests that can uncover subtle regressions or unintended behaviors that might be missed by manual testing or simple unit tests. The Scenario Director thus transforms the emulator from a mere playback device into a powerful,

instrumented testing platform, bringing the rigor of software engineering to the challenging domain of game logic and player experience validation.

The table below illustrates the workflow of the Emulator Scenario Director.

Phase	Component	Description	Role in Validation Loop
Define	`ScenarioSpec`	A JSON schema defining a complete test case: initial state, scripted inputs, telemetry signals, and assertions.	Provides a human-readable, editable format for defining complex gameplay tests [21].
Execute	Headless Emulator Harness (e.g., Conk64)	A Rust program that loads the emulator, runs the scenario deterministically, and captures telemetry.	Acts as the executor, running the test in a controlled, repeatable environment [5].
Assert	Assertion Engine	A component within the harness that evaluates captured telemetry against the `ScenarioSpec`'s assertions.	Determines pass/fail for the scenario, identifying specific logical failures.
Report	`ScenarioResult` / `CiFailure`	A structured JSON output from the harness, translated into a `CiFailure` by the CI pipeline on failure.	Provides the AI with clear, actionable feedback on why a gameplay test has failed [47].
Analyze	Telemetry Data Storage	The storage of successful scenario runs' telemetry data for analysis and property-based testing.	Enables data-driven analysis of gameplay systems and the creation of robust regression tests.

In essence, the Emulator Scenario Director completes the feedback loop for the entire development process. It ensures that not only is the code correct and the assets within budget, but the resulting game also behaves as intended. This capability is paramount for AI-driven development, as it provides the AI with a mechanism to test its own hypotheses about gameplay mechanics and receive empirical feedback, moving beyond simple code generation to true co-design and co-debugging of the interactive experience.

Synthesis and Strategic Implications of the Framework Architecture

The proposed AI-orchestrated development framework represents a mature and systematically engineered approach to integrating generative AI into a complex, safety-critical software development lifecycle. Its architecture is not a haphazard assembly of tools but a deeply interconnected ecosystem where each component serves a specific purpose within a coherent whole. The central thesis—that a "schema-first" methodology, grounded in a strict contract system between Rust types and JSON Schemas, can create a safe, predictable, and powerful development mode—is the framework's most significant

contribution [16] [55]. By mandating that all AI-visible actions be expressible as typed JSON structures and enforcing their validation in a CI gate, the system fundamentally constrains the AI's agency, channeling its creative potential into a verifiable and deterministic workflow [46]. This design choice directly confronts the primary challenges of Agentic AI, such as hallucination, unsafe actions, and lack of reproducibility, by building a robust, multi-layered defense-in-depth strategy [54] [60].

The framework's strength lies in the synergistic interplay of its core pillars. The **Technical Interoperability Layer**, built on `schemars` and `schema-guard`, establishes the foundational contract system that guarantees structural correctness at every turn [37]. The **AI Orchestration and Conditioning** layer, powered by the Knowledge Graph and a session-scoring engine, provides the AI with the necessary context and rules of engagement to operate effectively and responsibly [39] [53]. Finally, the three priority **Superpowers**—the Safe Patch Synthesizer, the Hardware-Aware Budget Planner, and the Emulator Scenario Director—serve as concrete, high-value applications of this architecture. They demonstrate how the abstract principles of schema-driven contracts translate into tangible capabilities that solve real-world problems in retro game development: preventing ROM corruption, managing hardware constraints, and validating gameplay logic [6] [20] [21]. The Knowledge Graph acts as the nervous system, connecting these disparate components and providing the AI with a holistic understanding of the project's capabilities and state [49]. Together, these elements create a closed-loop system where the AI can propose, the system can execute and validate, and the AI can learn and adapt from empirical feedback, embodying the ideal of a collaborative human-in-the-loop or agentic development partner [18] [52].

However, the framework's ambitious scope and technical sophistication introduce several strategic considerations and potential challenges. The most prominent is the inherent tension between the system's deep technical power and its ease of use. While the architecture is elegant and robust, its reliance on a custom-built ecosystem of Rust CLIs, extensive JSON schemas, a Knowledge Graph, and a complex CI workflow presents a significant barrier to entry [17]. The success of the project will depend heavily on the concurrent development of intuitive human-facing tools and visual editors that can abstract away this complexity [17]. Without accessible GUIs for editing `RomLayout`, `MissionDAG`, or `BudgetReport` files, the framework risks being relegated to a niche for expert Rust developers, undermining its goal of becoming a "first-class development mode" for a broader audience. Therefore, investing in UX/UI design from an early stage is not a secondary concern but a primary requirement for adoption.

Another challenge is the potential for performance bottlenecks within the iterative plan-and-execute loop. While deterministic and safe, the process of triggering a full CI run—which may involve rebuilding assets, running `starzip`, and launching a headless emulator—can be slow, potentially taking minutes per iteration [42]. A complex sequence of AI-driven refinements could lead to long development cycles if not managed carefully. Proactively addressing this is critical. The planned implementation of performance modeling and minimal rebuild logic (`nintendoor64-build rebuild --changed path`) is therefore a high-priority task [42]. Providing the AI with near-instantaneous feedback on the impact of its changes—such as which steps will rerun and an estimated incremental build time—is essential for maintaining productivity and keeping the development loop agile.

Finally, while the framework excels at verifying the *structure* and *execution* of AI proposals, the AI's internal *reasoning* process remains a "black box." Understanding *why* the AI chose a particular solution after receiving a `CiDigest` is non-trivial and will require sophisticated prompt engineering, interpretability techniques, or potentially reinforcement learning approaches. The system provides excellent feedback, but the AI's ability to synthesize that feedback into a novel and effective strategy is still a frontier of research [61].

Based on these considerations, a phased implementation strategy is recommended, mirroring the priorities outlined in the initial research plan. The immediate focus should be on establishing a rock-solid foundation: 1. **Lock in Schema Infrastructure:** Complete the integration of `schemars` across all core crates and deploy the `schema-guard` CI gate to cover all major contract types. This establishes the non-negotiable rulebook for the entire system [38]. 2. **Implement the N64 Vertical Slice:** Complete the end-to-end workflow for the N64, including the `sonia_bridge` for pre-flight checks, the `sonia-ai-n64-slice.yml` CI pipeline, and the wiring of `CiDigest` updates. This creates a proven, reusable template for the entire framework [57]. 3. **Enable Structured Feedback:** Implement the basic `session_conditioning` and `CiDigest` parsing logic to ensure the AI can receive structured feedback from CI failures, forming the basis of the adaptive loop [28]. 4. **Finalize the Knowledge Graph:** Draft and validate the full `features.sonia.json` manifest, meticulously mapping each "superpower" to its underlying system components and schemas. This builds the contextual intelligence layer that will empower the AI.

Once this stable foundation is in place, the framework can be extended incrementally to other platforms like PS1 and modern engines, and its capabilities can be expanded with more advanced features like the proactive budget planner and the node constellation

orchestrator. In conclusion, the proposed framework charts a clear and compelling course for the future of AI-assisted game development. By prioritizing safety, predictability, and verifiability through a rigorous schema-driven architecture, it offers a pragmatic and powerful pathway to unlocking the potential of generative AI while mitigating its inherent risks.

Reference

1. [PDF] SAP S/4HANA Cloud Public Edition 2602 - SAP Help Portal https://help.sap.com/doc/7c9e0bbbd1664c2581b2038a1c7ae4b3
2. EY Generative AI Use Cases repository - Scribd https://www.scribd.com/document/813848603/EY-Generative-AI-Use-Cases-repository
3. compact compressed textlist - Debian -- Packages https://packages.debian.org/bullseye/allpackages?format=txt.gz
4. Scaling & Automating Cyber Threat Intelligence (CTI) Operations ... https://www.researchgate.net/publication/385007252_Scaling_Automating_Cyber_Threat_Intelligence_CTI_Operations_with_Free_and_Open-source_Software_FOSS
5. FFplay media player - Ubuntu Manpage https://manpages.ubuntu.com/manpages/jammy/man1/ffplay-all.1.html
6. compact compressed textlist - Debian -- Packages https://packages.debian.org/bookworm/allpackages?format=txt.gz
7. TowardsDataScience-2023-博客中文翻译-三十八- - 绝不原创的飞龙 https://www.cnblogs.com/apachecn/p/18461365
8. https://mirrors.aliyun.com/openbsd/7.2/packages/am... https://mirrors.aliyun.com/openbsd/7.2/packages/amd64/SHA256
9. "sid" に含まれるソースパッケージ - Debian -- Packages https://packages.debian.org/ja/source/sid/allpackages
10. Compare Packages Between Distributions - DistroWatch.com https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=elive&secondlist=devuan&firstversions=0&secondversions=0&showall=yes

11. How would I approach "reflecting" a rust structure's schema into ... https://stackoverflow.com/questions/77835023/how-would-i-approach-reflecting-a-rust-structures-schema-into-json
12. awesome-lib/awesome-rust - Gitee https://gitee.com/awesome-lib/awesome-rust
13. https://packages.debian.org/bullseye/powerpcspe/al... https://packages.debian.org/bullseye/powerpcspe/allpackages?format=txt.gz
14. Escape Windows with Elementary OS | PDF | Desktop Environment https://www.scribd.com/document/650974257/The-Complete-Guide-to-Linux-1st-Edition-2023
15. Build an AI Chatbot Backend in Rust: Step-by-Step Tutorial https://dev.to/fortune-ndlovu/build-an-ai-chatbot-backend-in-rust-step-by-step-tutorial-4ap4
16. Programming by Chat: A Large-Scale Behavioral Analysis of ... - arXiv https://arxiv.org/html/2604.00436v1
17. (PDF) Programming by Chat: A Large-Scale Behavioral Analysis of ... https://www.researchgate.net/publication/403429855_Programming_by_Chat_A_Large-Scale_Behavioral_Analysis_of_11579_Real-World_AI-Assisted_IDE_Sessions
18. How to Build Human-in-the-Loop Plan-and-Execute AI Agents with ... https://www.marktechpost.com/2026/02/16/how-to-build-human-in-the-loop-plan-and-execute-ai-agents-with-explicit-user-approval-using-langgraph-and-streamlit/
19. 人工智能2026_4_6 - arXiv每日学术速递 http://www.arxivdaily.com/thread/78473
20. [PDF] Leveraging Procedural Audio for Greater Adaptability in Video Game https://escholarship.org/content/qt5wj1q3j2/qt5wj1q3j2_noSplash_acebdab5dd7deef5a298570f711f1976.pdf
21. A Diagnostic Taxonomy of Failure in Videogames https://dl.acm.org/doi/fullHtml/10.1145/3402942.3402979
22. Computational Intelligence in Data Science - Springer Nature https://link.springer.com/content/pdf/10.1007/978-3-031-16364-7.pdf
23. how to install a latest version of Nodejs on Ubuntu? [duplicate] https://stackoverflow.com/questions/74879973/how-to-install-a-latest-version-of-nodejs-on-ubuntu
24. (PDF) Social Media, Definition, and History - Academia.edu https://www.academia.edu/69506598/Social_Media_Definition_and_History
25. Read only access to all kubernetes objects in an EKS cluster https://stackoverflow.com/questions/76016679/read-only-access-to-all-kubernetes-objects-in-an-eks-cluster
26. Debian -- Software Packages in "bullseye" https://packages.debian.org/bullseye/kfreebsd-amd64/allpackages

27. Debian -- Software Packages in "sid", Subsection python https://packages.debian.org/en/sid/python/
28. Intermediate (200) – AWS Open Source Blog https://aws.amazon.com/blogs/opensource/category/learning-levels/intermediate-200/feed/
29. Arxiv今日论文| 2026-01-28 - 闲记算法 http://lonepatient.top/2026/01/28/arxiv_papers_2026-01-28
30. Proceedings of the 63rd Annual Meeting of the Association for ... https://aclanthology.org/volumes/2025.acl-long/
31. An Alternative Trajectory for Generative AI - arXiv https://arxiv.org/html/2603.14147v1
32. Download book PDF - Springer Nature https://link.springer.com/content/pdf/10.1007/3-540-45449-7.pdf
33. Sustainable EMIS: who is accountable? - UNESCO Digital Library https://unesdoc.unesco.org/ark:/48223/pf0000108603
34. [PDF] Proceedings of the 19th Linux Audio Conference - HAL https://hal.science/hal-05194352v1/file/proceedings.pdf
35. [PDF] ENTERPRISEOPS-GYM: ENVIRONMENTS AND EVAL - OpenReview https://openreview.net/pdf/259826bb08066d6575156ac53d2a100b8ef83d7a.pdf
36. Rust Full-Stack Development with AI Pair Programming - LinkedIn https://www.linkedin.com/posts/konstantinpupkov_my-book-is-now-published-after-months-activity-7416906813826109442-cURV
37. What's New - Oracle Linux Yum Server https://yum.oracle.com/whatsnew.html
38. Compare Packages Between Distributions - DistroWatch.com https://distrowatch.com/dwres.php?resource=compare-packages&firstlist=ubuntu&secondlist=debian&firstversions=6&secondversions=2&showall=yes
39. (PDF) Murdocca en - Academia.edu https://www.academia.edu/16307804/Murdocca_en
40. Profile for Ubuntu Inc. - Linknovate https://www.linknovate.com/affiliation/ubuntu-inc-5637112/all/?query=cross-layer%20program%20compilation
41. [PDF] ORCHESTRA: AI-Driven Microservices Architecture to Create ... https://aclanthology.org/2026.iwsds-1.18.pdf
42. Building AI Coding Agents for the Terminal: Scaffolding, Harness ... https://arxiv.org/html/2603.05344v1
43. Simon Willison on AI-generated open source software responsibility https://www.linkedin.com/posts/phil-jones-5043a81_my-answers-to-the-questions-i-posed-about-activity-7419707463345737728-us8y

44. GitHub Actions: How to pass toJSON() result to shell commands https://stackoverflow.com/questions/72953526/github-actions-how-to-pass-tojson-result-to-shell-commands
45. Integrating Go Test Execution Summary Reports in GitHub Actions https://www.linkedin.com/pulse/integrating-go-test-execution-summary-reports-github-actions-rana-hrvmc
46. GitHub Actions documentation https://docs.github.com/actions
47. Evaluate expressions in workflows and actions - GitHub Docs https://docs.github.com/en/enterprise-cloud@latest/actions/reference/workflows-and-actions/expressions
48. Workflow commands for GitHub Actions https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-commands
49. What is Model Context Protocol (MCP) and how does it work? https://www.linkedin.com/posts/bhabaniprusty_modelcontextprotocol-contextawareai-aiprotocols-activity-7362223193564110849-Nrlu
50. How to unit-test a bootloader on PIC18 with TCP/IP stack used? https://stackoverflow.com/questions/17379517/how-to-unit-test-a-bootloader-on-pic18-with-tcp-ip-stack-used
51. Arxiv今日论文| 2026-03-04 - 闲记算法 http://lonepatient.top/2026/03/04/arxiv_papers_2026-03-04
52. AgentOrchestra: Orchestrating Multi-Agent Intelligence with the Tool ... https://arxiv.org/html/2506.12508v5
53. 软件工程/系统与控制/多代理系统/数学软件2026_1_21 https://arxivdaily.com/thread/75878
54. AI Agents vs. Agentic AI: A Conceptual taxonomy, applications and ... https://www.sciencedirect.com/science/article/pii/S1566253525006712
55. Just-in-Time Historical State Reconstruction for Low-Latency ... - MDPI https://www.mdpi.com/2673-2688/7/4/117
56. Agentic AI: A Comprehensive Survey of Architectures, Applications ... https://www.researchgate.net/publication/397041422_Agentic_AI_A_Comprehensive_Survey_of_Architectures_Applications_and_Future_Directions
57. Anthropic's Claude Code Rebuilt from Behaviour in 1 Day - LinkedIn https://www.linkedin.com/posts/philipgreenwood_most-people-are-focusing-on-the-leak-that-activity-7446224788563333121-rSw8
58. Agentic AI: A Comprehensive Survey of Architectures, Applications ... https://arxiv.org/html/2510.25445v1
59. Computer Science - arXiv https://arxiv.org/list/cs/new

60. TRiSM for Agentic AI: A Review of Trust, Risk, and Security ... - arXiv https://arxiv.org/html/2506.04133v4
61. Computer Science - arXiv https://www.arxiv.org/list/cs/new?skip=875&show=1000
62. The Complete Full-Stack Developer Roadmap for 2026 https://dev.to/thebitforge/the-complete-full-stack-developer-roadmap-for-2026-2i0j